Euler and Navier-Stokes equations

- They model the dynamics of an incompressible fluid in a domain $\Omega \subseteq \mathbb{R}^3$.

- They are standardly written in terms of
  - the <u>velocity field</u>

$$u : \underset{\substack{\uparrow \\ t}}{\mathbb{R}^+} \times \underset{\substack{\uparrow \\ x}}{\Omega} \longrightarrow \underset{\substack{\uparrow \\ \text{vector field}}}{\mathbb{R}^3}$$

  - the <u>pressure field</u>

$$p : \mathbb{R}^+ \times \Omega \longrightarrow \mathbb{R} \quad (\text{scalar})$$

  - $\nu \geq 0$ the viscosity constant

$$(\nu \equiv 0 \rightsquigarrow \text{Euler}, \quad \nu > 0 \rightsquigarrow \text{NS})$$

$$(NS)_\nu \begin{cases} \partial_t u + (u \cdot \nabla) u \underbrace{- \nu \Delta u}_{\substack{\text{diffusivity} \\ \text{through viscous} \\ \text{forces}}} = - \nabla p \\ \operatorname{div} u = 0 \end{cases}$$

transport term
(u is transported by itself)

pressure forces, to maintain incompressibility

<u>Rmk</u> Since u is a vector field the first line of $(NS)_\nu$ is actually a system of 3 scalar equations.

If we write $u = (u_1, u_2, u_3)$ the three components of the vector field u, then

$$\begin{cases} \partial_t u_i + \sum_{j=1}^{3} u_j \partial_j u_i - \nu \Delta u_i = -\partial_i p \\ \sum_{i=1}^{3} \partial_i u_i = 0 \end{cases}$$

where $\partial_1, \partial_2, \partial_3$ refer to the partial derivatives w.r.t x_1, x_2, x_3 resp. (or x, y, z in standard notations), while we reserve ∂_t for derivatives w.r.t. time.

An important role for the dynamics is played by the <u>vorticity field</u>

$$\omega := \operatorname{curl} u = \nabla \times u$$

In coordinates:

$$\omega = (\partial_2 u_3 - \partial_3 u_2, \partial_3 u_1 - \partial_1 u_3, \partial_1 u_2 - \partial_2 u_1)$$

To derive the equation for ω, we simply take the curl of the equation.

- $\mathrm{curl}\,(\partial_t u) = \partial_t(\mathrm{curl}\, u) = \partial_t \omega$
- $\mathrm{curl}\,(\nu \Delta u) = \nu \Delta(\mathrm{curl}\, u) = \nu \Delta \omega$
- $\mathrm{curl}\,(-\nabla p) \equiv 0$ (curl ∇ always vanishes)
- it remains to evaluate $\mathrm{curl}((u\cdot\nabla)u)$.

For that purpose, notice first that

$$(u\cdot\nabla)u = \nabla\left(\frac{|u|^2}{2}\right) - u \times \omega$$

(this can be checked by expansion, e.g.)

and that for arbitrary vector fields A, B

$$\mathrm{curl}(A\times B) = A\,\mathrm{div}(B) - B\,\mathrm{div}(A) + (B\cdot\nabla)A - (A\cdot\nabla)B$$

(also checked by expansion)

Hence, taking into account that $\mathrm{div}(u) = 0 = \mathrm{div}(\omega)$:

$$\mathrm{curl}\,((u\cdot\nabla)u) = -\mathrm{curl}(u\times\omega) = (u\cdot\nabla)\omega - (\omega\cdot\nabla)u$$

In total:

$$\partial_t \omega + (u\cdot\nabla)\omega - (\omega\cdot\nabla)u - \nu\Delta\omega = 0$$

transport of ω by u — interpreted as a stretching of ω — diffusivity — no more pressure!

This equation has eliminated the pressure, but it involves both quantities u and ω. The velocity u is a function of ω though, through the Biot-Savart formula:

$$\begin{cases} \operatorname{curl} u = \omega \\ \operatorname{div} u = 0 \end{cases}$$

$$\hookrightarrow \underbrace{\operatorname{curl}(\operatorname{curl} u)}_{} = \operatorname{curl}\omega$$

for any vector field A,
$\operatorname{curl}(\operatorname{curl} A) = \nabla(\operatorname{div} A) - \Delta A$
hence here
$= \nabla \operatorname{div} u - \Delta u = -\Delta u$

In fine

$$
\begin{aligned}
u &= (-\Delta)^{-1}(\operatorname{curl} \omega) \\
&= \operatorname{curl}\left((-\Delta)^{-1}\omega\right) \\
&= \operatorname{curl}\left(\frac{1}{4\pi}\int_\Omega \frac{\omega(y)}{|x-y|}\,dy\right) \\
&= \frac{1}{4\pi}\int_\Omega \frac{\omega(y)\times(x-y)}{|x-y|^3}\,dy
\end{aligned}
$$

which we shall simply write $u = BS(\omega)$.

The situation greatly simplifies in the case of 2D flows (i.e. assuming $\Omega = \Omega_{2d} \times \mathbb{R}$ and that $u_3 \equiv 0$ and u_1, u_2 only depend on x and y).

In such case:

1) $\omega = (\omega_1, \omega_2, \omega_3) = (0, 0, \omega_3)$

is aligned with the z axis, so that ω can be understood as a <u>scalar</u> quantity

$$\omega_3 := \partial_1 u_2 - \partial_2 u_1$$

(abuse of notation, should be clear from context)

2) The stretching term identically vanishes. Indeed,

$$(\omega \cdot \nabla) u = \omega_3 \partial_3 u = \omega_3 \cdot \vec{0} = \vec{0}$$

since u does not depend on z.

3)
$$\begin{aligned} u &= \operatorname{curl}\left((-\Delta)^{-1}\omega\right) \\ &= \operatorname{curl}\left((0, 0, (-\Delta^{-1})\omega_3)\right) \\ &= \left(-\partial_2(\Delta^{-1}\omega_3), \partial_1(\Delta^{-1}\omega_3), 0\right) \\ &= \Big(\underbrace{\nabla^{\perp}(\Delta^{-1}\omega)}_{\in \mathbb{R}^2}, 0\Big) \end{aligned}$$

where $\nabla^{\perp} := (-\partial_2, \partial_1)$.

In the sequel since $\omega = (0, 0, \omega_3)$ in this case we shall simply write ω instead of ω_3 in 2D, and consider it as a scalar field. Introducing the <u>stream function</u> $\psi := (\Delta^{-1})(\omega)$ we obtain the vorticity/stream function formulation in 2D!

$$(*) \quad \begin{cases} \partial_t \omega + (\nabla^\perp \psi \cdot \nabla)\omega - \nu \Delta \omega = 0 \\ \Delta \psi = \omega \end{cases}$$

and $u = \nabla^\perp \psi$.

The relation $u = \nabla^\perp \psi$ implies that the velocities of the particles of the fluid, i.e. u, is everywhere tangent to the level curves of the fct ψ (indeed, $\nabla \psi$ is orthogonal to the level curves of ψ so $\nabla^\perp \psi$ is tangent to them). That is the reason for its name, the stream function.

For a 2D manifold without boundary, such as the sphere S^2, the Navier-Stokes equations keep the form $(*)$, where Δ has to be understood as the Laplace-Beltrami operator. The velocity/pressure formulation has to be adapted instead, this reflects the difficulty to work with vector fields rather

than scalar quantities.

The interested reader will find account on these in the nice papers from the 70's and 80's

(1) A.A. Ilyin & A.N. Filatov
Unique solvability of the Navier-Stokes equations on a two-dimensional sphere
Doklady Akademii Nauk SSSR, (1988) Vol 301, N°1, p 18-22

(available freely on www.mathnet.ru, very short and to the point, but in russian. Contains the Coriolis force variant)

(2) D. G. Ebin & J. Marsden
Groups of diffeomorphisms and the motion of an incompressible fluid
Annals of Mathematics Vol 92 (1970) p. 102-163

(see also Marsden, Ebin, Fischer 1972 for highly valuable lecture notes Google for MaEbFi1972.pdf)

FEM discretization of Navier-Stokes 2D

NS9

We shall only work with the vorticity / stream function formulation, due to its simplicity of treatment (relative to the (u,p) formulation) for surface meshes.

Note first that in the first equation of $(*)$, we may rewrite

$$(\nabla^\perp \psi \cdot \nabla)\omega = \operatorname{div}(\omega \nabla^\perp \psi) - \omega \cdot \operatorname{div}(\nabla^\perp \psi)$$
$$= \operatorname{div}(\omega \nabla^\perp \psi)$$

since $\operatorname{div}(\nabla^\perp \cdot)$ identically vanishes.

Multiplying the first equation of $(*)$ by some (smooth and time independent) test function φ, we obtain, after integrating by parts the resulting identity:

$$\boxed{\partial_t \int \omega \varphi - \int \omega \nabla^\perp \psi \cdot \nabla \varphi + \nu \int \nabla \omega \cdot \nabla \varphi = 0}$$

We use a FEM discretization (in space only) NS10

and write

$$\omega_h = \sum_{i=1}^{N} \Omega_i(t)\, \varphi_i(x) \qquad \left(N \equiv n^{th} \text{ degrees of freedom}\right)$$

$$\psi_h = \sum_{i=1}^{N} \Psi_i(t)\, \varphi_i(x)$$

where $\varphi_1, \dots, \varphi_N$ refer to the shape functions of the FEM, and $\Omega_i(t)$ and $\Psi_i(t)$ are the corresponding (time dependent) coefficients.

<u>Rmk</u> Note that we could have used different elements for ω and ψ, here we sticked to the simplest choice, which shall actually be $\mathbb{P}_1$-Lagrange for both.

Taking as test function $\varphi = \varphi_k$, for an arbitrary $1 \le k \le N$ yields

$$\sum_{i=1}^{N} \Omega_i'(t) \int \varphi_i \varphi_k + \nu \sum_{i=1}^{N} \Omega_i(t) \int \nabla\varphi_i \cdot \nabla\varphi_k$$

$$= \sum_{i,j=1}^{N} \Omega_i(t)\, \Psi_j(t) \int \varphi_i\, \nabla^{\perp}\varphi_j \cdot \nabla\varphi_k$$

We use a compact notation

$$\Omega(t) := \begin{pmatrix} \Omega_1(t) \\ \vdots \\ \Omega_N(t) \end{pmatrix} \qquad \underline{\psi}(t) := \begin{pmatrix} \psi_1(t) \\ \vdots \\ \psi_N(t) \end{pmatrix}$$

$$M = \left(\int \varphi_i \varphi_j \right)_{1 \le i,j \le N} \qquad S = \left(\int \nabla \varphi_i \cdot \nabla \varphi_j \right)_{1 \le i,j \le N}$$

and have therefore

$$M \Omega'(t) + \nu S \Omega(t) = T(\Omega(t), \psi(t))$$

where the bilinear transport term T is given by

$$T(\Omega(t), \psi(t))_k = \sum_{i,j=1}^{N} \Omega_i(t) \psi_j(t) \int \varphi_i \nabla^{\perp} \varphi_j \cdot \nabla \varphi_k$$

Let us let aside for a short moment the actual computation and storage of T (we won't use a 3-tensor but do each term on the fly).

We now discretize Ω and Ψ in time and set

$$\underset{\mathbb{R}^N}{\underset{\cap}{\Omega^m}} := \Omega(m\Delta t), \quad \underset{\mathbb{R}^N}{\underset{\cap}{\Psi^m}} := \Psi(m\Delta t)$$

where $\Delta t > 0$ is fixed (in practice it should by linked to the space discretization size h for stability issues) and $m \in \mathbb{N}$ is the iterate number.

We replace $\Omega'(t)$ by $\dfrac{\Omega(t+\Delta t) - \Omega(t)}{\Delta t}$

and therefore end-up with the implicit scheme

$$\boxed{\begin{cases} (M + \nu \Delta t S)\Omega^{m+1} = M\Omega^m + \Delta t \cdot T(\Omega^m, \Psi^m) \\ S\Psi^{m+1} = -M\Omega^m \end{cases}}$$

(the second equation is the FEM Poisson solver which we already studied)

which allows to obtain $(\Omega^{m+1}, \Psi^{m+1})$ given (Ω^m, Ψ^m) through solving 2 linear systems

with matrices $M+\nu\Delta t S$ and S respectively. Both are symmetric, $M+\nu\Delta t S$ is SPD, S is SPD if restricted to the orthogonal of its kernel of dimension 1 (corresp to constant functions).

NSB

<u>Rmk</u>

We might have prefered the (apparently easier) discretization

$$M\Omega^{m+1} = M\Omega^{m} - \nu\Delta t\, S\Omega^{m} + \Delta t\, T(\Omega^{m}, \psi^{m})$$

but (as it would be for the heat equation) this would have terrible stability properties

(the right-hand side involve S which is a "large" (in norm) matrix compared to M, while having S on the left-hand side in our implicit scheme makes it damper oscillations and increase stability).

[a longer discussion on CFL conditions might be in order TODO]

It remains to deal with the computation of $T(\Omega^m, \underline{\psi}^m)$.

It only has to be computed one per time iteration (the reason why we don't rely on a 3-tensor alike the mass of stiffness matrices (2-tensors)).

We use the classical assembly method to decompose the term through a sum of integrals over all triangles of the mesh:

$$\sum_{i,j=1}^{N} \Omega_i^m \psi_j^m \int_{Mesh} \varphi_i \nabla^\perp \varphi_j \cdot \nabla \varphi_k$$

$$= \sum_{i,j=1}^{N} \Omega_i^m \psi_j^m \sum_{T \in \mathcal{T}} \int_T \varphi_i \nabla^\perp \varphi_j \cdot \nabla \varphi_k$$

$$= \sum_{T \in \mathcal{T}} \underbrace{\sum_{i,j=1}^{N} \Omega_i^m \psi_j^m \int_T \varphi_i \nabla^\perp \varphi_j \cdot \nabla \varphi_k}_{\text{if } T = (i_A, i_B, i_C)\text{, only need to consider cases s.t. } i,j,k \in \{i_A, i_B, i_C\}\text{, integral being 0 if not.}}$$

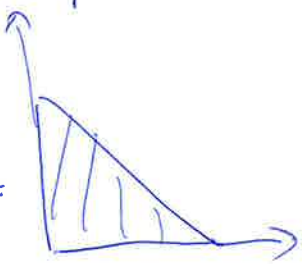
NSIS

To compute

$$\int_T \varphi_i \nabla^\perp \varphi_j \cdot \nabla \varphi_k$$

for $i, j, k \in \{i_A, i_B, i_C\}$,

very similar to the mass and stiffness base computation on the reference triangle

$\hat{T} =$ 

$0 \leq s, t \leq 1$

$s + t \leq 1$

For $\mathbb{P}_1$, $\nabla \varphi_k$ and $\nabla^\perp \varphi_j$ are C^{st} vectors, and φ_i is an affine scalar fct

$\longrightarrow$ easy :-)